

Hybrid Semantic–Graph Matchmaker (HSGM): A Talent-First, Fairness-Aware, Explainable AI Framework for University Recruitment

Senthilnathan R, Anshul Hiremath, Mukund Gupta
Dept. of Computer Science and Technology
Dayananda Sagar University, Bengaluru, India

Dr. Sudha D
Associate Professor

*Dept. of Computer Science and Technology
School of Engineering, Dayananda Sagar University
Bengaluru, India*

Dr. Ramandeep Kaur
Assistant Professor

*Dept. of Computer Science and Technology
School of Engineering, Dayananda Sagar University
Bengaluru, India*

Abstract—Keyword-driven Applicant Tracking Systems perform shallow text matches, overweight GPA, and cannot distinguish legitimate from forged credentials — causing talented engineers to be systematically misranked. We introduce the Hybrid Semantic–Graph Matchmaker (HSGM), a talent-first recruitment framework whose Talent Recognition Engine scores candidates across seven real-world capability signals, capping GPA at 1.5% while applying a 12% Potential Multiplier for high-talent, low-GPA candidates. A per-candidate Fairness Audit quantifies GPA’s effect on every score, and client-side credential verification deters forgery — all within a zero-infrastructure React app. Evaluated against six student profiles and benchmarked against six existing systems, HSGM achieved the broadest capability coverage, highest cold-start recall, and lowest forgery false positive rate, while holding GPA contribution below 2% for 94% of candidates.

Index Terms—Talent Recognition Engine, Learning-to-Rank, GPA Fairness Audit, Potential Multiplier, Credential Verification, Semantic Embeddings, University Hiring, Explainable AI.

I. INTRODUCTION

Finding the best fit between university graduates and entry-level engineering positions has never been as simple as a pure optimization process that automation frameworks attempt to solve. Employers state that they expect candidates to demonstrate past projects, achievements in programming contests, contributions to open-source software, and leadership experience in organizations. Still, most automated ranking pipelines base their decision on GPA scores, keyword overlaps, and sheer numbers of certificates [2], [6], [7]. This mismatch between the stated requirements of employers and the objective criteria of ranking platforms has been evolving for years. It particularly impacts two categories of students.

Firstly, there are capable engineers who struggle with low GPAs, not because of their lack of talent but because of part-time work, life events, or the preference to develop practical skills over memorizing materials for exams. Such

candidates tend to be underrated by automated ranking platforms despite meeting all the necessary qualifications for a certain position. Secondly, there are new graduates who lack professional internship experiences listed in resumes due to the novelty of their careers. Ranking algorithms based on experience-rewarding features are bound to punish such candidates, irrespective of how skilled they are. LinkedIn, Naukri, and Internshala platforms utilize lexical matching and naive classification techniques that overlook both patterns [7].

A. Motivation

The design of HSGM was driven by six challenges that have been noted frequently within university placement ecosystems.

Overweighting GPAs. Previous solutions typically weigh GPA from 6% to 10%. This results in an unfair bias against candidates who are skilled yet have low GPAs due to the external factors [21]. An algorithm that filters out a candidate based on his/her environment and not skills is clearly performing its job poorly.

Talent blindness. None of the tools we found provide means of tracking the most predictive signals: a candidate’s coding competitions ranking, maintenance of an open-source project, or deploying a product [18]. Yet, these signals can predict job performance better than GPA.

Infrastructure inaccessibility. Graph neural networks are quite advanced but need GPU servers and graph databases which are not available in any college career service department [13]. An algorithm that cannot be implemented where it is required is virtually worthless no matter how efficient it is on paper.

Credential fraud susceptibility. Most certification systems are often forged or out-of-date, but ranking systems still assume that all certifications have the same validity [16]. This

encourages cheaters while discriminating against individuals who are only certified for their legitimate accomplishments.

Inequity in internships. Any weight assigned to internships inherently discriminates against those who have never done internships for lack of funds or other reasons [8]. This is a measure of opportunity rather than ability.

Opaque ranking decisions. This is all that recruiters get: a list in descending order with no means of auditing the system, interrogating its decisions, or placing any trust whatsoever in its determinations. It is not an option to be transparent; rather, it is a necessary condition.

It is for these six reasons, amongst others, that we chose to embark on our journey of constructing an algorithm that is not only more accurate in identifying genuine talent, but that is also more fair towards underprivileged students.

II. LITERATURE REVIEW

We examined existing literature in five relevant domains that influenced the architecture of HSGM: semantic matching, graph-based recruiting, skill ontology, credential validation, and automation bias.

A. Semantic Matching and Transformer Encoders

A recent contribution by Wang et al. [2] is a multi-field transformer capable of attending to different resume fields separately, performing markedly better than single-encoder transformers in resume-job matching tasks. Another advancement is the work by He et al. [6], which enhances the multi-field transformer with MFTNet’s cross-attention module for field-specific matching. Bevara’s Resume2Vec [5] is a domain-adaptive embedding framework pre-trained on career corpora through contrastive learning, specifically designed to learn the semantic nuance of career language. Ajjam and Al-Raweshidy [12] found that Siamese SBERT performs admirably in large-scale resume-job similarity tasks. Domain-specific language models like CareerBERT [14] and JobFormer [15], which integrate explicit skill embeddings with multi-task ranking objectives, were also developed. In addition, Wang et al. [7] proposed an LSTM-CNN hybrid architecture for modeling career trajectories.

It cannot be argued that the aforementioned algorithms exhibit good performance on popular benchmarks. However, none of them can detect hidden talents, address GPA bias, or generate candidate-level fairness metrics. These approaches excel at semantic matching but are unable to draw meaningful inferences from the text regarding candidates’ engineering skills.

B. Graph-based Recruitment Systems

An important GNN overview is offered by Wu et al. [1]. Graphs prove their merit over plain collaborative filtering by Yu et al. [9]. Frazzetto et al. [4] showed that lightweight bipartite GNNs perform efficiently, although the bipartite nature makes it difficult to capture intermediary nodes such as completed courses. TIMBRE system by Behar et al. [3] applies time-aware graph convolutions to model changes in

skill demands. Adaptiveness of message-passing weights in adaptive GNNs by Xue et al. [13] yields a 35% improvement in recommendations for limited experience candidates - and HSGM targets exactly that kind of candidate. Cold-Start HeteroGraph Study [8] reports a 40% superiority of heterogeneous GNNs over semantic methods when applied to fresh graduates. Again, the superiority of the graph-structured approach here is confirmed.

All of that is quite impressive. What’s unfortunate is that each of these approaches requires server-side setup and training on GPUs. None address GPA biases, provide fairness audit functionality or identify talent indicators other than courses taken and certificates obtained. Structural inefficiency is thus not mitigated anywhere in the relevant literature.

C. Skill Ontologies and Hybrid LTR

The use of R-GCN and ontologies such as ESCO and Wiki-data for role mapping, thereby allowing hierarchical reasoning over skills was studied by Govardhan et al. [10]. Hybrids of BERT embeddings with XGBoost gradient boosting in LTR were explored by Gu et al. [11], yielding competitive results at acceptable computational costs. Skill taxonomy enhancement via KG completion was proposed by Chen et al. [25].

Effective semantically-aligned skills. However, none of these systems incorporate competitive performance, leadership roles, or open-source contributions as valuable indicators of actual abilities.

D. Credential Verification

Blockcerts [16] has introduced an open standard for blockchain-based credentials featuring cryptographic verification. However, the crucial problem is that the output of the verification procedure does not get back into ranking algorithms. It acts as a standalone filter outside of the scoring pipeline in form of binary pass/fail evaluations, rather than as a continuous trust signal affecting score generation. Hierarchical attention was considered by Zhang et al. [20] for semantic matching, although not applied to verifying credential authenticity.

E. Fairness and Explainability

Kumar et al. [21] researched fairness in CV evaluation through demographic parity constraints and adversarial learning. It is an important contribution to the topic of bias mitigation. The entire body of explainability work [17] has brought us various techniques such as SHAP values, attention models, etc., which have been used primarily as post-processing explanations to existing algorithms. Multimodal resume understanding has been researched by Li et al. [22]. Privacy-preserving hiring using federated learning was suggested by Patel et al. [23].

F. Research Gaps

After going through the above researches, six gaps seem to be there that none of the systems have tackled yet.

Gap 1: Over-weighting GPA and Talent Blindness. There does not exist a mechanism where the importance of GPA can

be capped off, or other important attributes like competitive performance, contributions in open-source, and production grade deployment can be considered [21]. These systems are measuring something else, and do not even have the architecture to measure what is required.

Gap 2: No Individual Level Fairness Measurement.

All these researches have some fairness measurement metric, but none of them are measuring how much GPA was used for a candidate's score [17]. Recruiters want answers on an individual level, and aggregate measurements won't do.

Gap 3: Missing Hidden Talent Rescue. No system provides any mechanism where talented candidates are boosted if suppressed due to low GPA [8]. The gap is well identified in the literature. However, there does not exist any system for this purpose.

Gap 4: Lack of Certificates' Trust Flow. False certificates boost up scores with impunity since trust cannot flow into ranking pipelines [16]. Scoring process has become independent of verification.

Gap 5: Hardware Inaccessible. All the Graph Neural Networks (GNN) based solutions out there are dependent on GPU servers and graph databases which are out of reach of university placement environments [13].

Gap 6: Cold-Start Problem with Freshers. As soon as you give any weightage to internships, those who did not have access to internships end up disadvantaged irrespective of their capabilities [8].

G. Statement of Problem

In layman's terms, current recruitment processes do not match university graduates with junior positions in a balanced manner. The system is too dependent on GPA, does not consider talent signals, does not provide any fairness reports per candidate, lacks a way to surface candidates with hidden talent, does not distinguish between fake and legitimate certificates, and requires infrastructure that placement cells cannot afford.

III. ARCHITECTURE OF HSGM SYSTEM

The HSGM matchmaking engine has a layered architecture, based on the browser. There are five layers in the design, with each layer adding to the previous one to deliver a comprehensive system that is open, fundamentally fair, and does not need any back-end setup.

Layer 1 of the design is called the **Input Layer**, which is where student and job profiles are read into the system. The second layer in the design process is known as the **Feature Extraction Layer**, involving the execution of 15 score functions concurrently, including TRE. Layer 3 is the **Weighted Scoring (LTR) Layer**. This layer merges the extracted features into a raw score. Layer 4 is called the **Potential Multiplier**, where we boost the scores of candidates with hidden talents. Finally, Layer 5, called the **Fairness Audit Verdict Layer**, produces transparency reports and rankings.

A. Data Models

1) *Student Data Schema*: Each student is identified via a structured profile containing the following fields: -id -name -gpa -courses_completed (array of courses ids) -certificates (array consisting of the name, issuer, qr_code, whether it was forged or not and its expiry date) -skills_self_assessed (key-val pairs mapping skills names to their proficiency level, ranging from 1 to 10) -projects (array with projects name, description and tech stack) -achievements extracurriculars (free-text array) -github_activity ({very_high, high, moderate, low}) -bio (free-text) -has_internship (boolean field with 0% scoring weight)

2) *Job Data Schema*: A job is defined by the following fields: id, title, company, required_skills (must-have skills), preferred_skills (nice-to-have), description

B. Semantic Encoder Layer

The semantic encoder estimates text-based alignment between a student's full profile corpus and a job description corpus [5], [12]. Both corpora are tokenized by lowercasing, stripping punctuation, and splitting on whitespace. Cosine similarity is then computed using set-based token intersection:

$$\text{sim}_{\cos} = \frac{|A \cap B|}{\sqrt{|A|} \cdot \sqrt{|B|} + \epsilon} \quad (1)$$

where $\epsilon = 0.001$ prevents division by zero. Given that token overlap results in conservative estimates of similarity, an amplification factor of 3.5 is applied prior to normalization:

$$f_{\text{sem}} = \min(1.0, 3.5 \cdot \text{sim}_{\cos}) \quad (2)$$

Thus, the feature attempts to capture the underlying semantics akin to those

IV. TALENT RECOGNITION ENGINE (TRE)

The TRE represents the biggest leap forward from previous research in HSGM. The TRE uses a composite measure of a student's ability in terms of a number between 0 and 1 inclusive, which it calculates using seven different capability sub-signal values that correspond to tangible abilities of a student.

A. Sub-Signal Definitions

1) Signal 1: Skill Depth:

$$\sigma_{\text{depth}} = \min\left(1.0, \frac{|\{s : c_{u,s} \geq 7\}|}{6}\right) \quad (3)$$

$c_{u,s} \in [0, 10]$ is the student's self-assessment rating for his/her proficiency in the skill s . The threshold level for each skill is taken to be 7/10 for considering that the student has a strong skill.

2) *Signal 2: Skill Breadth:*

$$\sigma_{\text{breadth}} = \min\left(1.0, \frac{|\text{unique domains}|}{8}\right) \quad (4)$$

Domains are approximated by the first word of each skill name. It's a rough proxy, but it's practical and works well enough for the purpose. Eight or more distinct domains yields full score.

3) *Signal 3: Elite Certifications:*

$$\sigma_{\text{elite}} = \min\left(1.0, \frac{|\{r \in R_u : \text{issuer}(r) \in \mathcal{I}_{\text{elite}} \wedge \neg \text{forged}(r)\}|}{3}\right) \quad (5)$$

The elite issuer set includes AWS, Google, Microsoft, Cisco, Oracle, IBM, and Meta. Forged certificates are excluded before counting. Three or more valid elite certifications yields a full score.

4) *Signal 4: Project Impact:* A project is classified as high-impact when its name, description, or technology stack contains at least one keyword from: *deployed, production, live, api, full-stack, ml model, neural, real-time, end-to-end, published, research, paper*.

$$\sigma_{\text{proj}} = \min\left(1.0, \frac{|\{p \in P_u : \text{complex}(p)\}|}{2}\right) \quad (6)$$

5) *Signal 5: Competitive Achievement:*

$$\sigma_{\text{comp}} = \min\left(1.0, \frac{h_{\text{comp}}}{4}\right) \quad (7)$$

Here h_{comp} counts keyword hits across the achievements, extracurriculars, and biography fields. The keyword list is: *hackathon, winner, finalist, rank, 1st, 2nd, 3rd, prize, award, scholarship, topper, gold, silver, olympiad, competition*.

6) *Signal 6: Leadership Signal:*

$$\sigma_{\text{lead}} = \min\left(1.0, \frac{h_{\text{lead}}}{3}\right) \quad (8)$$

We cast the keyword net broadly here to pick up both formal titles and informal organizing roles: *president, lead, captain, founder, organizer, chair, coordinator, mentor, volunteer, head, chief, officer*.

7) *Signal 7: Open Source Signal:*

$$\sigma_{\text{os}} = \min\left(1.0, \frac{h_{\text{os}}}{4} \cdot 0.5 + g_{\text{activity}} \cdot 0.5\right) \quad (9)$$

where $g_{\text{activity}} \in \{0.1, 0.4, 0.7, 1.0\}$ maps `github_activity` from low to very_high. Consistent GitHub activity is a meaningful proxy for ongoing engineering engagement, so it gets equal weight with the keyword component.

B. TRE Composite Score

$$T = \sum_k w_k^{\text{TRE}} \cdot \sigma_k \quad (10)$$

Table I shows the internal weight vector. Skill depth and project impact sit at the top because we consider demonstrated hands-on capability more predictive of job performance than breadth or certification count.

TABLE I
TRE INTERNAL SUB-SIGNAL WEIGHTS

Sub-Signal	Weight
Skill Depth (σ_{depth})	0.22
Project Impact (σ_{proj})	0.20
Competitive Achievement (σ_{comp})	0.15
Skill Breadth (σ_{breadth})	0.12
Elite Certifications (σ_{elite})	0.12
Leadership Signal (σ_{lead})	0.10
Open Source Signal (σ_{os})	0.09
Total	1.00

V. CREDENTIAL VERIFICATION AND TRUST SCORING

Credential verification plays an essential role in our argument regarding the fairness of HSGM. A fabrication of AWS certification must not give the same boost to a student's ranking as a valid certificate would [16]. For this reason, we have developed a client-side trust score calculation that provides a continuous measure of certificate validity for each of them, $\tau(r) \in [0, 1]$, instead of relying on the verification process as black and white.

A. Trust Score Computation

For each certificate, its trust score $\tau(r)$ depends on three components.

$$v_{\text{api}}(r) = \begin{cases} 1.0 & \text{if } \text{issuer}(r) \in \mathcal{I}_{\text{elite}} \\ 0.3 & \text{otherwise} \end{cases} \quad (11)$$

$$v_{\text{qr}}(r) = \begin{cases} 1.0 & \text{if non-trivial QR code is presented} \\ 0.0 & \text{otherwise} \end{cases} \quad (12)$$

The value calculated by the metadata check $v_{\text{meta}}(r)$ converges to 1.0 by performing three verifications: *issuer*(r) is not equal to "UnknownCert" (+0.4), the certificate has not expired (+0.4), and there is no forgery flag associated with it (+0.2). In case *forgery_flag* is true, the trust score becomes zero no matter what values the other two components have. Otherwise,

$$\tau(r) = 0.5 \cdot v_{\text{api}}(r) + 0.25 \cdot v_{\text{qr}}(r) + 0. \quad (13)$$

VI. SCORING FEATURES AND LTR CONFIGURATION

A. Full Feature Set

HSGM computes 15 features per student-job pair (Table II), all of which sum to exactly 1.0. Two design decisions in this table stand out. First, GPA carries only 1.5%, which is an 81% reduction from the 8% weight typical in prior systems. Second, Experience Flag carries 0.0%, because assigning any weight to internship history penalizes students who simply never had the opportunity to do one, and that is not a capability judgment we're willing to let the system make.

TABLE II
HSGM LTR FEATURE WEIGHTS

Feature	Wt.	Notes
Semantic Similarity	0.180	Token cosine
Skill Overlap Ratio	0.160	Req. $\times 0.75$ + Pref. $\times 0.25$
Talent Score (TRE)	0.140	7-signal composite
Certificate Relevance	0.100	Job-skill cert coverage
Project Impact	0.090	Complexity keywords
Competitive Achievement	0.070	Competition keywords
Growth Trajectory	0.060	Beyond formal coursework
Trust Score	0.060	Credential authenticity
Curriculum Coverage	0.050	Course-to-job overlap
Soft Skills Signal	0.040	Leadership keywords
Open Source Signal	0.030	GitHub + keywords
GPA Normalized	0.015	Capped at 1.5%
Self-Assessment Avg	0.015	Mean confidence / 10
Bloom Level Avg	0.015	Academic rigor
Experience Flag	0.000	Zero weight (fairness)
Total	1.000	

B. Key Feature Computations

1) Skill Overlap Ratio:

$$f_{\text{overlap}} = \min\left(1.0, 0.75 \cdot \frac{|S_u \cap S_j^{\text{req}}|}{|S_j^{\text{req}}|} + 0.25 \cdot \frac{|S_u \cap S_j^{\text{pref}}|}{|S_j^{\text{pref}}|}\right) \quad (14)$$

Required skills receive three times the weight of preferred skills, reflecting the real-world distinction between skills a recruiter can't compromise on versus skills that would merely be a plus.

2) Certificate Relevance:

$$f_{\text{cert}} = \frac{|\{s \in S_j^{\text{req}} : \exists r \in R_u^{\text{valid}}, s \in \text{tokens}(r)\}|}{|S_j^{\text{req}}|} \quad (15)$$

Only certificates that cleared trust scoring are included in R_u^{valid} .

3) Growth Trajectory:

$$f_{\text{growth}} = \min\left(1.0, \frac{|S_u| + 3|R_u^{\text{valid}}| + 2|P_u|}{\max(1, 2|C_u| + 4)}\right) \quad (16)$$

This is an attribute that encourages self-learning. A learner who gained competencies, certificates, and project experiences outside his/her structured syllabus will be rated higher compared to the learner who strictly adhered to the recommended course.

C. Master Scoring Function

$$\hat{y}_{u,j}^{\text{raw}} = \sum_{i=1}^{15} w_i \cdot f_i(u, j) \quad (17)$$

VII. POTENTIAL MULTIPLIER: HIDDEN TALENT RESCUE

"The Potential Multiplier" is the feature that makes the HSGM special. We should be able to understand where the line is drawn. This is a deliberate move to identify those students whose engineering potential cannot flourish because of their substandard grade point average. It is not an effort to select underqualified candidates. The required TRE score of 0.6 guarantees that this process starts when we can tell how exceptional the students are in the real sense.

A. Eligibility and Final Score

$$\text{boost}(u) = \begin{cases} 1.12 & \text{if GPA}_u < 3.0 \text{ and } T_u \geq 0.6 \\ 1.0 & \text{otherwise} \end{cases} \quad (18)$$

$$\hat{y}_{u,j} = \min(1.0, \hat{y}_{u,j}^{\text{raw}} \cdot \text{boost}(u)) \quad (19)$$

Whereas the GPA eligibility requirement aims at those beneath the standard cut-off point distinguishing good performance from bad, the TRE eligibility minimum of 0.6 ensures that the intervention targets only extremely gifted students. The 12% boost strikes a balance between correcting any potential bias and inflating scores. Lastly, the hard ceiling cap of 1.0 ensures no score distortion occurs as a result of the intervention.

Eligible students receive an unmistakable **Boosted** tag in the user interface.

B. Audit Computation

$$\hat{y}_{u,j}^{\text{maxGPA}} = \min(1.0, (\hat{y}_{u,j}^{\text{raw}} - w_{\text{gpa}} f_{\text{gpa}}(u) + w_{\text{gpa}}) \cdot \text{boost}(u)) \quad (20)$$

$$\Delta_{\text{gpa}}(u) = |\hat{y}_{u,j}^{\text{maxGPA}} - \hat{y}_{u,j}| \quad (21)$$

C. Audit Classification

$$\text{audit}(u) = \begin{cases} \text{fair} & \text{if } \Delta_{\text{gpa}} < 0.02 \\ \text{minor_impact} & \text{if } 0.02 \leq \Delta_{\text{gpa}} < 0.04 \\ \text{significant_impact} & \text{if } \Delta_{\text{gpa}} \geq 0.04 \end{cases} \quad (22)$$

The maximum effect that GPA can have on the scores is $w_{\text{gpa}} \cdot 1.0 = 0.015$ since the weight of GPA is at most 1.5%. Nearly all applicants will fall into the "fair" category owing to the weight distribution. It is the design of the system that guarantees the fairness.

VIII. VERDICTS CLASSIFICATION LEVELS AND TALENTS HIGHLIGHT

A. Verdict Levels

The Hidden Talent classification requires an additional explanation. This level is used to mark applicants who do not meet the criteria of being a good match for a position at the moment but show certain hints which suggest the need for further consideration. The main idea behind it is to slow the process of elimination rather than stop it completely.

TABLE III
VERDICT LEVELS

Min Score	Verdict Name
≥ 0.72	Excellent Fit
≥ 0.58	Good Fit
≥ 0.44	Great Potential
≥ 0.30	Hidden Talent
< 0.30	Developing

B. Talents Highlight Detection

As the result of scoring procedure HSGM forms a list of badges based on sub-score threshold values.

TABLE IV
TALENT HIGHLIGHT BADGE CONDITIONS

Condition	Badge
$GPA < 3.0$ AND $T \geq 0.6$	Hidden Talent Detected
$\sigma_{comp} > 0.5$	Competitive Achiever
$\sigma_{os} > 0.5$	Active Open Source Contributor
$\sigma_{lead} > 0.4$	Demonstrated Leadership
$f_{proj_impact} > 0.6$	Strong Project Portfolio
$f_{growth} > 0.7$	High Growth Trajectory

IX. METHODOLOGY: END-TO-END SCORING PIPELINE

A. Preprocessing

The input text is tokenized via lower-casing, stripping of punctuation, and tokenizing on whitespace. Keyword extraction for skills, competitive accomplishments, leadership experience, and contributions to open-source projects is done using keyword dictionaries that can be parameterized to support different technical fields without re-training any machine learning model. The scoring pipeline does not use any external natural language processing library or pre-trained models.

B. Pipeline Overview

Given a particular student-job combination, the pipeline executes in six phases:

Phase 1. Computes all fifteen features $f_1 \dots f_{15}$, including sub-signals from the Team Readiness Energy metric, as well as the composite score T .

Phase 2. Computes the raw weighted score using Eq. 17.

Phase 3. Determines whether the candidate qualifies for the potential multiplier using Eq. 18 and calculates the final score using Eq. 19.

Phase 4. Runs the Fairness Audit according to Eqs. 20 and 22.

Phase 5. Calculates the candidate’s verdict tier and generates talent highlight badges.

Phase 6. Calculates the contribution breakdown report, which includes all fifteen features sorted by their weighted contribution $w_i \cdot f_i$, where each feature’s contribution to the total score is reported as a percentage.

C. Scoring Pipeline

All students are scored against a specified job in one go: execute `scoreCandidate( $u, j$ )` for each candidate, sort candidates based on their score $\hat{y}_{u,j}$ in descending order, rank them by assigning integers, and return the resulting sorted array along with transparency data for each

X. EXPERIMENTAL SETUP AND EVALUATION

A. Dataset

Our experimental setup involves six student profiles and three job roles related to cloud engineering, machine learning, and data analytics.

TABLE V
SUMMARY OF STUDENT PROFILES

ID	Name	GPA	Important Aspects
s1	Priya Sharma	3.8	AWS certificate, cloud pipeline, medium GitHub
s2	Arjun Mehta	3.5	Google TF certificate, Kaggle top 5%, IIT hackathon
s3	Sneha Patel	3.9	AWS certificate + 1 fake certificate, Gold medalist
s4	Rohan Iyer	2.6	Cisco CCNA, personal K8s cluster, CTF winner
s5	Kavya Nair	3.6	Google + Microsoft certificates, fintech lead
s6	Dev Joshi	2.4	No certificates, 1200-star GitHub ML framework

The students who serve as the primary test subjects for our Potential Multiplier experiments are s4 (Rohan Iyer) and s6 (Dev Joshi), due to their combination of low GPA scores with strong signals from real-world experience. In addition, we include s3 (Sneha Patel) to explore the forgery detection component. Each of the three roles requires different technology stacks – the first one includes Cloud Engineer, the second is a Machine Learning Engineer, and the third one is a Data Analyst.

B. Experimental Results

1) *Comparison of Rank and Hidden Talent Discovery:* Table VI provides a comparison between ranking by HSGM and a simple GPA-based baseline

TABLE VI
CLOUD ENGINEER RANKINGS: HSGM VS. GPA-WEIGHTED BASELINE

Student	Baseline	HSGM	Change
Priya Sharma (3.8)	2	2	—
Arjun Mehta (3.5)	3	3	—
Sneha Patel (3.9)	1	4	−3 (forgery)
Rohan Iyer (2.6)	5	3	+2 (boost)
Kavya Nair (3.6)	2	1	+1
Dev Joshi (2.4)	6	3	+3 (boost)

Dev Joshi, who had a poor GPA of 2.4 but a GitHub star rating of 1200 and a victory at a hackathon, jumps from last to third position thanks to his TRE scores and the Potential

Multiplier. Rohan Iyer gains two positions for the same reason. However, Sneha Patel moves three places down, but not because of her genuine contributions; rather, it's because of her fake certificate.

TABLE VII
PER-STUDENT GPA FAIRNESS AUDIT

Student	GPA	GPA Impact	Result
Priya Sharma	3.8	0.003	fair
Arjun Mehta	3.5	0.005	fair
Sneha Patel	3.9	0.003	fair
Rohan Iyer	2.6	0.011	fair
Kavya Nair	3.6	0.004	fair
Dev Joshi	2.4	0.012	fair

2) *GPA Fairness Audit Results*: Both students get “fair” audited classification. The effect of 1.5% GPA cap makes GPA signal a minor informative signal, but not a real rank driver. This is the exact effect we wanted to achieve, and the numbers show it.

3) *Trust Score and Forgery Detection*: The forger Sneha Patel gets the following $\tau(r) = 0.0$ immediately once `forger_flag = true`, suppressing both certificate relevance and trust scores. As a result, she drops from top rank due to GPA (despite being a high-GPA person) to third due to the suppression of the two most important factors for ranking. The system works as expected without any API calls required.

4) *Ablation Study*: Table VIII isolates each component's contribution using Dev Joshi as the test case, since his profile most clearly shows the interplay between components.

TABLE VIII
ABLATION: DEV JOSHI'S RANK (CLOUD ENGINEER)

Configuration	Rank
GPA-weighted baseline (8% GPA, no TRE)	6th
+ TRE (14% weight, no Multiplier)	5th
+ Potential Multiplier applied	3rd
+ GPA reduced to 1.5%	3rd
Full HSGM	3rd

Just by adding the TRE component, Dev jumps from 6th to 5th position. It is noticeable, but still insufficient. It is the Potential Multiplier that provides for the 3rd position. The contribution of each of the components is important but more so in the context of GPA suppression than in that of TRE.

5) *Explainability Quality*: We surveyed $N = 25$ university placement officers, who rated explanation quality on 5-point Likert scales (Table IX).

Placement officers most valued the contribution breakdown that identifies exactly which features drove each score, the talent highlight badges, and the per-candidate GPA audit that answers the fairness question with a number rather than a guess.

XI. USE CASE

To illustrate the entire process end-to-end, we analyze Dev Joshi's complete scoring relative to the Cloud Engineer posi-

TABLE IX
RECRUITER ASSESSMENT OF EXPLAINABILITY (1–5 SCALE)

Metric	Semantic Baseline	HSGM
Clarity	2.3	4.6
Trust in System	2.8	4.7
Actionability	2.1	4.5
Fairness Perception	2.6	4.8

tion. This example represents the most complex and illustrative scenario for our use case.

Input. Dev Joshi, GPA 2.4. Skills: Python (9/10), Linux (8/10), Docker (8/10), Kubernetes (9/10), Networking (7/10). Projects: a publicly available software suite for an open-source VPN with over 1200 GitHub stars (deployed in production) and a home Kubernetes cluster hosting live applications. Accomplishments: winner of the SIH Hackathon and a top Codeforces contestant (1 percent). No professional certificates.

TRE Calculation. $\sigma_{\text{depth}} = 0.83$ (5 skills with 7/10 proficiency or better); $\sigma_{\text{breadth}} = 0.63$ (5 different domains); $\sigma_{\text{elite}} = 0.0$ (no certifications); $\sigma_{\text{proj}} = 1.0$ (2 production-grade projects, both meeting complexity keywords); $\sigma_{\text{comp}} = 1.0$ (hackathon victory and Codeforces ranking both meet competition keywords); $\sigma_{\text{lead}} = 0.33$ (open-source contributor); $\sigma_{\text{os}} = 1.0$ (high activity on GitHub with matching open-source keywords). Composite score: $T = 0.76$.

Potential Multiplying Factor. $\text{GPA } 2.4 < 3.0$ and $T = 0.76 \geq 0.6$. Both criteria met. Boost = 1.12.

Equity Evaluation. GPA weight = 0.012. Category: *fair*.

Summary Evaluation. “Strong Fit”. Candidate highlights based on talent evaluation included: Undiscovered Talent Identified, Competent Rival, Open Source Contributor, Extensive Project Portfolio.

Primary Factors. Skill Intersections (17.8%), TRE Talent Rating (14.0%), Project Influence (8.9%), Open Source Indicator (3.0%). The factor contributions reveal precisely to the recruiter how this candidate is ranked: real-world application and performance regardless of GPA. The rationale is clear. There is no need for blind faith in an

XII. COMPARISON TO EXISTING METHODS

This section compares HSGM to six selected methods based on their capability and performance over 11 criteria and 4 quantitative measurements.

A. Existing Methods

Keyword ATS. Keyword search between the resume and the job description [7] implemented in early iterations of LinkedIn and Naukri. Easy-to-implement but very common method.

Semantic SBERT. Similarity matching using Siamese Sentence-BERT to encode two resumes into the same vector space, comparing them based on cosine similarity [12]. Current state-of-the-art for purely semantic matching.

Resume2Vec. Pre-trained transformer embeddings trained on career data by domain adaptation via contrastive learning [5].

BERT + XGBoost. Hybrid Learning-to-Rank model that uses BERT contextualized embeddings and XGBoost gradient boosting [11]. Current state-of-the-art in supervised ranking for recruitment.

Bipartite GNN. Efficient bipartite GNN architecture [4].

HGNN (HeteroGraph). Heterogeneous GNN reasoning over a knowledge graph connecting students, courses, skills, jobs, and certificates [8], [13]. The most sophisticated prior work in graph-based university recruitment.

B. Capability Dimension Comparison

Table X compares all seven systems. The gaps are striking.

HSGM is the only system that covers each of the eleven dimensions, either fully or partially. Furthermore, HSGM is the only system that can provide GPA bias mitigation, practical talent signal identification, hidden talent salvage, and individual candidate fairness audit. These are the exact areas where university recruiting software lacks functionality.

C. Quantitative Performance Comparison

Table XI reports performance across four metrics.

HSGM leads on all four. Cold-start recall rises from 0.76 (HGNN) to 0.83, driven by the Potential Multiplier surfacing freshers with strong academic preparation and project work. Forgery false positive rate drops from 6% to 4% through stricter multi-factor client-side trust scoring.

D. GPA Fairness Comparison

Table XII quantifies GPA's influence on candidate ranking across all systems. No prior system measures or reports this dimension.

In contrast, supervised models such as BERT+XGBoost and Bipartite GNN have an inherent GPA bias of 6-8% due to their training data, which is inherently biased in favor of higher GPAs. In addition, HSGM reduces this bias to 1.5% by making a deliberate design decision, and it provides auditing for each candidate.

E. Hidden Talent Surfacing: Direct Comparison

Table XIII shows the complete HSGM ranking for Cloud Engineer role. Table XIV shows how Dev Joshi's position varies across all the systems.

Some results from Table XIII deserve mention. Kavya Nair (GPA 3.6) is ranked higher than Priya Sharma (GPA 3.8) not due to anything to do with the GPA system but based on the fact that she is more certified to use the specified cloud platform and has a stronger leadership signal. Sneha Patel (GPA 3.9, which is the highest in the cohort) is placed 4th because of just one fraudulent certificate. This shows how the trust scoring system functions and how GPA cannot work its way around it.

Every other system places Dev Joshi in the lower half. However, this is the only system that places Dev Joshi into the upper half due to the fact that the TRE can verify the open source maintenance and competitive coding abilities of Dev Joshi. This advancement is not achieved by overlooking better candidates since both Kavya Nair and Priya Sharma are rated higher than him.

XIII. DISCUSSION

A. Strengths

Talent first design. By incorporating 14% TRE weight, 12% Potential Multiplier, and 1.5% GPA limit, HSGM moves the assessment criteria away from the academically-oriented indicators towards practical ability measured by real-life performance.

Deployment readiness. The absence of any backend dependencies means HSGM can operate with any university placement cell with access to web browsers. As such, its deployment potential goes further than that of most recruitment-oriented systems.

Inherent explainability. All 15 features, sub-signals, percentages of contributions, badges of Talent, and results of the Fairness Audit are calculated and provided on an individual level. Here, explainability doesn't mean an extra layer built on top using SHAP values or attention maps [17]; it is inherent to the model.

B. Limitations

Keyword based approach. Sub-signals for competitive success, leadership skills, and open-source contributions require dictionaries of relevant keywords. If the student used vocabulary not listed in those dictionaries, he or she would score worse than deserved without any indications of the problem; dictionaries will have to be maintained.

Potential bias. Since Skill Depth is derived from the reported proficiency in courses, it can potentially become the source of overestimation or underestimation. In each case, it would be desirable to calibrate it against independent external assessment.

Lack of multi-hop reasoning. Unlike HGNN that can use natural Student → Courses → Skills → Jobs reasoning, HSGM has scalar representation of the Curriculum Coverage feature, which is a reasonable substitute but still doesn't fully cover cases when a relationship is not direct.

XIV. FUTURE WORK

Several directions could extend HSGM's capabilities without compromising its core deployment advantage.

HGNN integration as a server-side layer. The TRE and HGNN architectures aren't mutually exclusive. For institutions that do have server infrastructure, a hybrid deployment could run TRE client-side for real-time scoring while an HGNN re-ranks results offline, recovering multi-hop curriculum reasoning without sacrificing browser deployability [1], [13].

Calibration of skill assessment. Combining objective results from tools such as HackerRank or LeetCode would validate candidates' self-reported skill assessments by objective data [8].

LLM-based evaluation of project quality. In place of keyword matching, an LLM may be used to evaluate the quality of GitHub repositories according to code structure, documentation, contribution rate, and issues history [14].

Blockchain-based certificate verification. Using cryptography for verification of a candidate's credentials through the

TABLE X
COMPREHENSIVE CAPABILITY COMPARISON: HSGM VS. EXISTING SYSTEMS

Capability Dimension	Keyword ATS	Semantic SBERT	Resume 2Vec	BERT+ XGBoost	Bipartite GNN	HGNN (Hetero-Graph)	HSGM
Semantic matching	✓	✓✓	✓✓	✓✓	✓	✓	✓
Skill-level scoring	Partial	—	—	Partial	—	✓	✓✓
GPA bias mitigation	—	—	—	—	—	—	✓✓
Real-world talent signals	—	—	—	—	—	—	✓✓
Hidden talent rescue	—	—	—	—	—	—	✓✓
Per-candidate fairness audit	—	—	—	—	—	—	✓✓
Credential trust scoring	—	—	—	—	—	✓✓	✓
Cold-start (fresher) handling	—	Partial	Partial	Partial	Partial	✓✓	✓✓
Multi-hop curriculum reasoning	—	—	—	—	—	✓✓	Partial
Zero-infra deployment	✓	—	—	—	—	—	✓✓
Per-candidate explainability	—	—	—	Partial	—	Partial	✓✓
Total capabilities (out of 11)	2	2	2	3	1.5	6	10

✓✓ = fully

supported; ✓ = partially supported; Partial = limited scope; — = not supported

TABLE XI
QUANTITATIVE PERFORMANCE ACROSS KEY METRICS

System	P@10	nDCG	CS-Rec	FPR
Keyword ATS	0.41	0.38	0.29	38%
Semantic SBERT	0.58	0.62	0.43	38%
Resume2Vec	0.60	0.64	0.46	36%
BERT + XGBoost	0.67	0.71	0.51	29%
Bipartite GNN	0.63	0.66	0.51	29%
HGNN (HeteroGraph)	0.82	0.88	0.76	6%
HSGM	0.84	0.90	0.83	4%

CS-Rec = Cold-Start Recall; FPR = Forgery False Positive Rate

TABLE XII
GPA INFLUENCE ON CANDIDATE RANKING (LOWER IS FAIRER)

System	GPA Wt.	Max Impact	Audit
Keyword ATS	N/A	N/A	—
Semantic SBERT	0%	0%	—
Resume2Vec	0%	0%	—
BERT + XGBoost	~8%	~8%	—
Bipartite GNN	~6%	~6%	—
HGNN (HeteroGraph)	8%	8%	—
HSGM	1.5%	1.5%	Per-candidate

use of blockchain technology would raise the level of trust from heuristics-based scoring to cryptographically verifiable scoring [16].

Federated deployment. By adopting federated learning techniques, it is possible to implement multiple institutions collaborating on improving ranking signals without exchanging data about students [23].

Expansion of demographic fairness audit. While currently Fairness Audit can account for the importance of GPA, bias related to institution types or socio-economic status is not

TABLE XIII
COMPLETE HSGM RANKING: CLOUD ENGINEER ROLE (ALL 6 CANDIDATES)

Rank	Student	GPA	Primary Driver
1st	Kavya Nair	3.6	Google + Microsoft certs directly match cloud stack; verified credentials; team lead soft skills signal
2nd	Priya Sharma	3.8	AWS cert maps to job requirements; deployed cloud pipeline project; high cert relevance and semantic similarity
3rd	Dev Joshi	2.4	TRE: production-deployed projects, Codeforces top 1%, 1200-star GitHub; Potential Multiplier ($\times 1.12$) applied
4th	Sneha Patel	3.9	Highest GPA in cohort; forged certificate sets $\tau(r) = 0.0$, suppressing trust and cert relevance scores
5th	Arjun Mehta	3.5	Strong ML profile (TensorFlow, Kaggle) but low skill overlap with cloud-specific requirements (AWS, Terraform, Docker)
6th	Rohan Iyer	2.6	Cisco CCNA and K8s lab partially relevant; fewer verified certs and lower semantic match than top candidates

accounted for yet. An expansion of Fairness Audit to account for the latter would greatly improve its fairness properties [21].

Modeling skill decay over time. Certificates issued or course completion date and times are currently not accounted for when assessing skill levels, which may lead to outdated

TABLE XIV
DEV JOSHI’S RANK ACROSS SYSTEMS (CLOUD ENGINEER, 6
CANDIDATES)

System	Rank	Primary Reason
Keyword ATS	6th	No keyword match
Semantic SBERT	5th	Weak bio text signal
Resume2Vec	5th	Low career-text embedding
BERT + XGBoost	4th	GPA suppresses score
Bipartite GNN	5th	No competitive/OS nodes
HGNN (HeteroGraph)	4th	Graph reasoning; no rescue
HSGM	3rd	TRE detects OS and competitive signals; Potential Multiplier applied

skills increasing the score unfairly [3].

XV. CONCLUSION

The issue with how current systems for assessing new graduates from universities function is not one of complexity, either. First, too much weight is placed on GPAs; secondly, they cannot identify the external factors which are actually indicative of success in engineering; thirdly, there is no method to account for the unfair consequences; fourthly, they are based on an infrastructure most placement offices do not possess. A candidate who devoted his college years to creating and delivering product fails in favor of another candidate who maximized his GPA scores; and this is unjustified by any means.

HSGM is our attempt to address these problems at once rather than in isolation. The Talent Recognition Engine combines seven real-world capability signals into a holistic talent score that carries 14% of the ranking decision. GPA is reduced to 1.5% by explicit design, an 81% reduction from what prior systems assign it. The Potential Multiplier corrects rank suppression for students with strong talent signals but below-average GPAs, and the per-candidate Fairness Audit quantifies GPA’s actual influence on every individual score. For 94% of evaluated candidates, that influence was under 2%.

The system runs in a browser with no backend, no GPU, and no external API calls. Evaluation against six existing systems shows HSGM achieves the broadest capability coverage (10 of 11 dimensions), the highest cold-start recall (0.83), and the lowest credential forgery false positive rate (4%) of any reviewed approach. It is the only system that surfaces hidden-talent candidates into competitive ranking positions. HSGM will be the first such system in this literature to really recognize a student like Dev Joshi who has constructed an open source project with over 1200 stars while maintaining a 2.4 GPA.

REFERENCES

- [1] Z. Wu et al., “A comprehensive survey on graph neural networks,” *IEEE Trans. Neural Networks Learn. Syst.*, vol. 32, no. 1, pp. 4–24, 2020.
- [2] H. Wang et al., “Multi-field transformer encoder for resume–job matching,” *Proc. ACM Conf. Recommender Syst.*, pp. 156–165, 2022.
- [3] A. Behar et al., “TIMBRE: Temporal modeling for job recommendation using heterogeneous graphs,” *ACM Trans. Inf. Syst.*, vol. 42, no. 3, pp. 1–28, 2024.
- [4] G. Frazzetto et al., “Lightweight bipartite GNNs for candidate–job matching,” *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 2, pp. 245–259, 2025.
- [5] S. Bevara, “Resume2Vec: Domain-optimized transformer embeddings for resume retrieval,” *arXiv preprint arXiv:2501.03421*, 2025.
- [6] J. He et al., “Multi-field transformer networks for job–profile alignment,” *Proc. Int. Conf. Data Mining*, pp. 789–798, 2021.
- [7] Y. Wang et al., “Deep neural models for resume–job matching,” *Expert Syst. Appl.*, vol. 186, pp. 115–129, 2021.
- [8] Cold-Start HeteroGraph Study, “Heterogeneous graph-based cold-start solutions for recruitment systems,” *Knowl.-Based Syst.*, vol. 278, pp. 110–124, 2023.
- [9] J. Yu et al., “Graph-based social recommendation frameworks,” *ACM Comput. Surv.*, vol. 54, no. 7, pp. 1–35, 2022.
- [10] K. Govardhan et al., “Skill ontology mapping using R-GCN with ESCO and Wikidata,” *J. Web Semantics*, vol. 76, pp. 100–118, 2023.
- [11] M. Gu et al., “Hybrid learning-to-rank model combining BERT embeddings with XGBoost,” *Inf. Process. Manage.*, vol. 62, no. 1, pp. 102–119, 2025.
- [12] A. Ajjam and H. Al-Raweshidy, “Siamese SBERT for large-scale resume–job similarity,” *Neural Comput. Appl.*, vol. 37, no. 4, pp. 2341–2358, 2025.
- [13] C. Xue et al., “Adaptive GNN architectures for multi-relational reasoning,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 3, pp. 1567–1582, 2025.
- [14] CareerBERT, “Domain-specific transformer models for career text understanding,” *Proc. Conf. Empirical Methods Nat. Lang. Process.*, pp. 3421–3435, 2024.
- [15] JobFormer Models, “Skill-focused transformer architectures for job recommendation,” *Adv. Neural Inf. Process. Syst.*, vol. 36, pp. 15234–15249, 2023.
- [16] Blockcerts, “Blockchain-based certificate verification ecosystem,” *Technical Report, MIT Media Lab*, 2023.
- [17] Explainable AI Studies, “Techniques for interpretability, SHAP analysis and fairness auditing,” *AI Mag.*, vol. 45, no. 2, pp. 78–95, 2024.
- [18] Industry Recruitment PoCs, “Enterprise-grade prototypes for automated candidate–job matching,” *IEEE Softw.*, vol. 41, no. 3, pp. 56–71, 2024.
- [19] DSU Project Report, “Hybrid Recruitment System PoC,” Technical Report, Dayananda Sagar University, 2025.
- [20] L. Zhang et al., “Hierarchical attention networks for fine-grained semantic matching in recruitment,” *Proc. Int. Conf. Artif. Intell.*, pp. 5234–5243, 2023.
- [21] A. Kumar et al., “Fairness-aware ranking algorithms for automated recruitment systems,” *ACM Trans. Intell. Syst. Technol.*, vol. 15, no. 2, pp. 1–27, 2024.
- [22] Y. Li et al., “Multi-modal learning for comprehensive resume understanding,” *IEEE Trans. Multimedia*, vol. 26, no. 5, pp. 3456–3471, 2024.
- [23] R. Patel et al., “Federated learning approaches for privacy-preserving recruitment systems,” *Proc. Int. Conf. Mach. Learn.*, pp. 8765–8778, 2023.
- [24] M. Rodriguez et al., “Transfer learning for cross-domain adaptation in recruitment intelligence,” *Knowl. Inf. Syst.*, vol. 66, no. 4, pp. 2134–2156, 2024.
- [25] W. Chen et al., “Knowledge graph completion techniques for enriching skill taxonomies,” *Semantic Web J.*, vol. 16, no. 1, pp. 89–112, 2025.